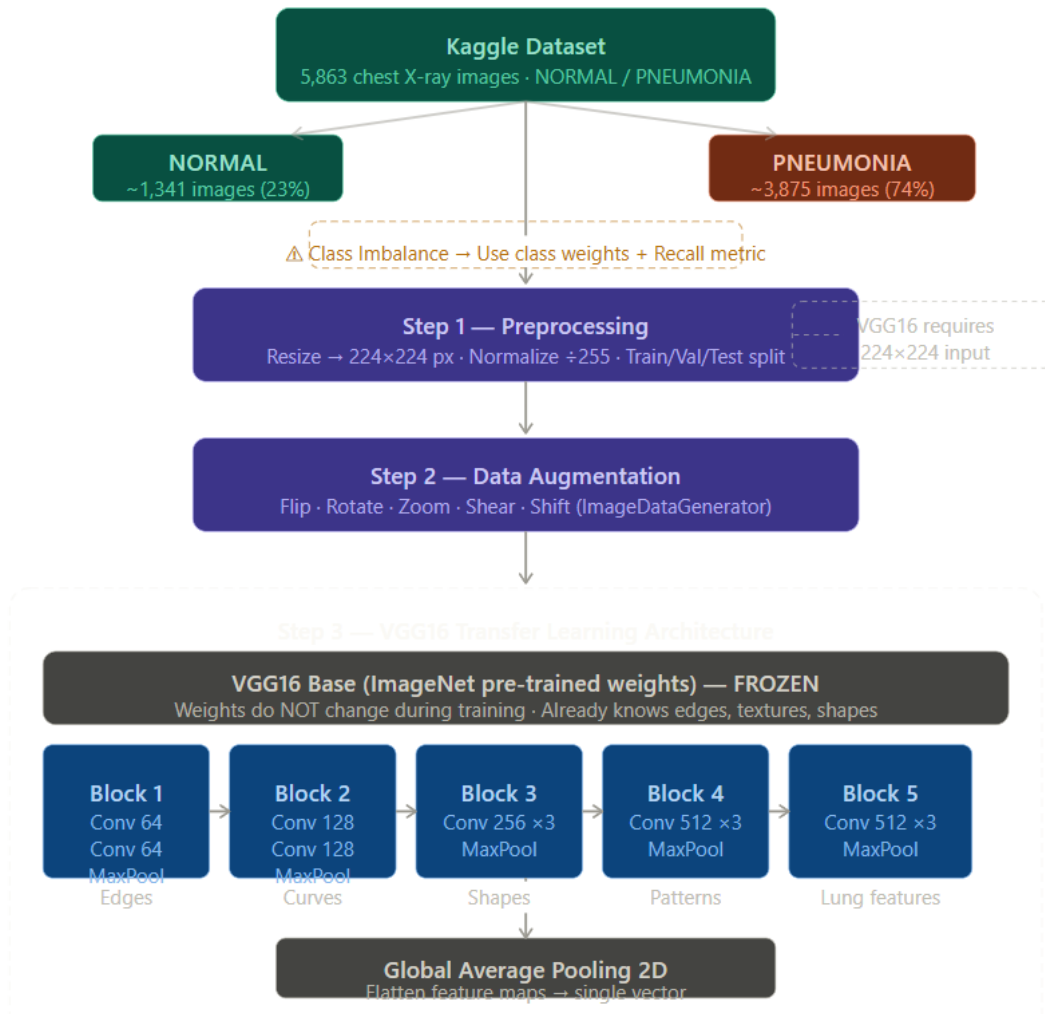


Project Full Pipeline -



Training Loop –



Steps Wise Training of Each image

Step 1 — Input: one batch of X-ray images enters

Training doesn't use all 5,216 images at once. We split them into batches of 32. The model sees batch 1, learns from it, then batch 2, and so on. One full pass through all batches = 1 epoch.

Batch of 32 images



P = Pneumonia N = Normal

Each image: 224×224×3 pixels · Label: 0 or 1

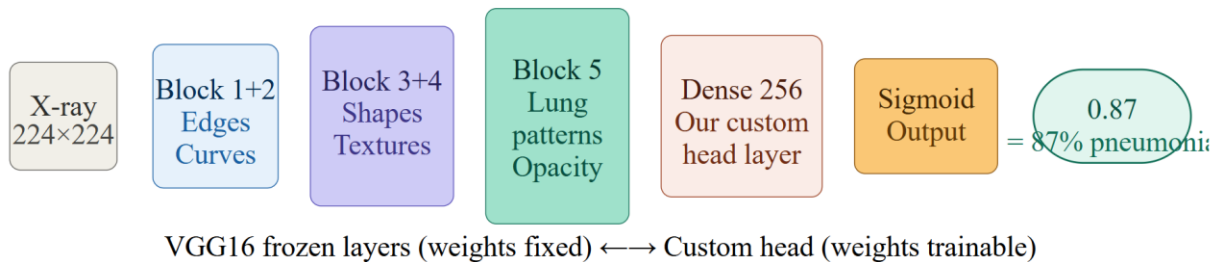
VGG16 model
Waiting to process batch

163 batches per epoch · 20 epochs total

Why batch size 32? If we used 1 image at a time, weight updates are very noisy. If we used all 5,216 at once, we need huge memory. 32 is the sweet spot — stable gradients, fits in GPU memory, trains well.

Step 2 — Forward pass: image travels through VGG16 layers

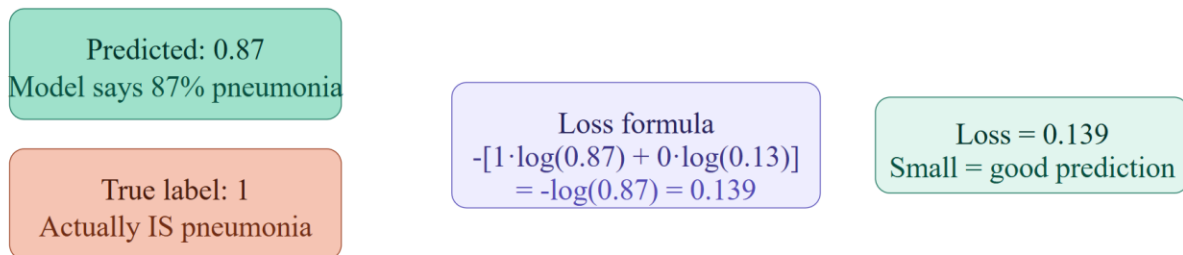
Every pixel value passes through 16 layers of VGG16. Each layer transforms the data — early layers detect edges, middle layers detect shapes, deep layers detect high-level patterns like 'lung opacity'. At the end, a single probability number comes out.



Key insight: The VGG16 base is FROZEN — its weights don't change. It's a feature extractor. Only the Dense 256 layer and final sigmoid learn new weights for pneumonia detection. This is why it trains fast even on a small dataset.

Step 3 — Loss calculation: how wrong were we?

The model predicted 0.87 (87% pneumonia). The true label is 1 (definitely pneumonia). Binary Cross-Entropy compares these and gives a loss score — a number representing how wrong the prediction is.



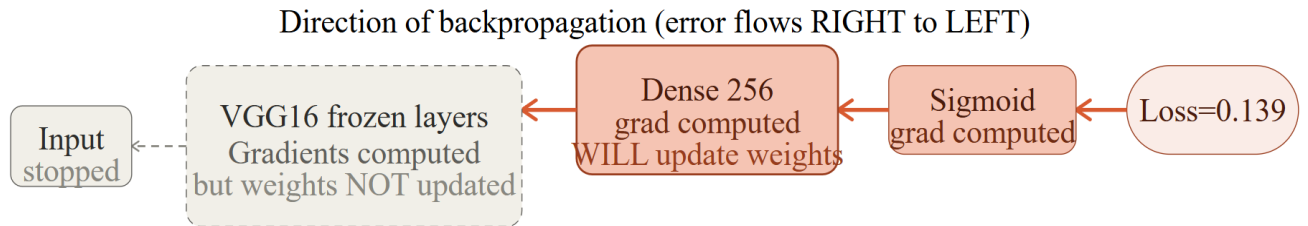
Wrong + confident example, pred=0.05 → Loss = 3.0 (huge!)

The more confident and more wrong, the bigger the penalty

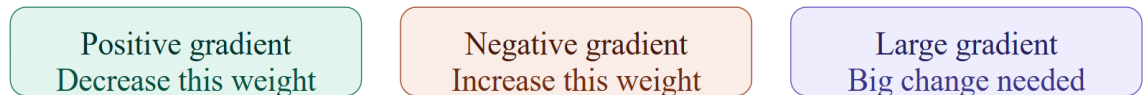
Why this matters in medicine: Predicting 'Normal' (0.05) when the patient has Pneumonia gives a loss of 3.0. Predicting 'Pneumonia' (0.87) correctly gives 0.139. The loss function punishes dangerous mistakes very heavily, guiding the model to avoid them.

Step 4 — Backpropagation: trace which weights caused the error

Now we work backwards. Using calculus (chain rule), we calculate how much each weight in the custom head layers contributed to the loss. This gives us a 'gradient' for each weight — a number saying 'increase me' or 'decrease me' and by how much.



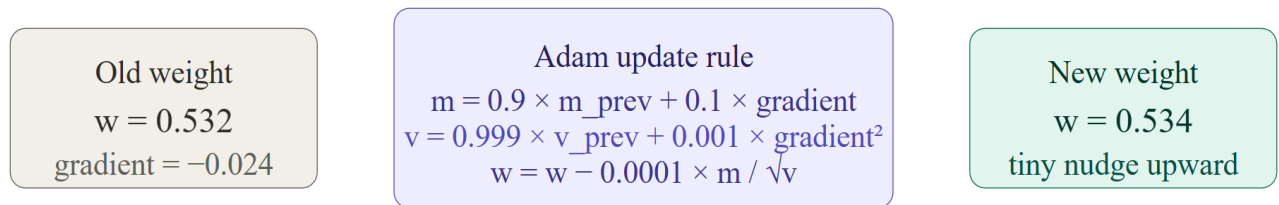
What a gradient tells us for each weight:



Chain rule in simple terms: Think of a chain of people passing blame. The sigmoid says 'Dense layer, you're 70% responsible'. Dense says 'VGG block 5, you're responsible'. But VGG is frozen — it hears the blame but ignores it. Only Dense updates.

Step 5 — Weight update: Adam optimizer adjusts the weights

Now Adam uses the gradients to nudge each weight slightly in the direction that reduces loss. The learning rate (0.0001) controls how big each nudge is. Very small — because we don't want to ruin the pre-trained VGG16 knowledge.



Adam's two smart components:

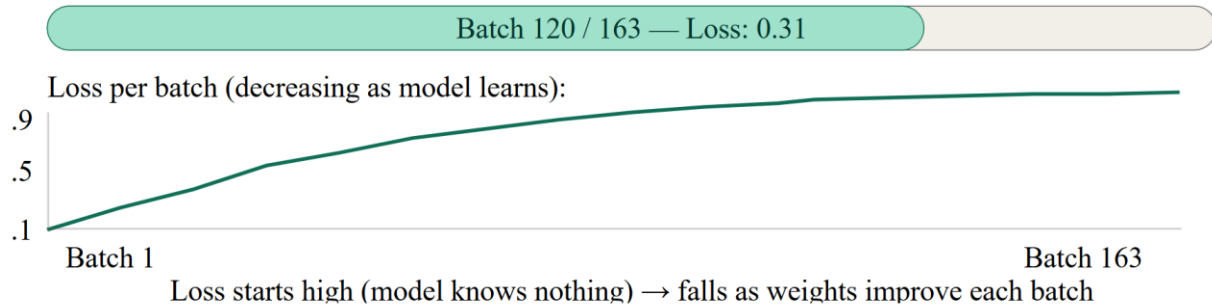


Why learning rate 0.0001 (very small)? VGG16's weights were trained on 14 million images. A large learning rate (like 0.01) would destroy that knowledge in a few updates. Small rate = careful, surgical adjustments to the custom head only.

Step 6 — One epoch: repeating this for all 163 batches

Steps 1–5 happen for every batch. With 5,216 training images and batch size 32, there are 163 batches per epoch. After all 163 batches complete, one epoch is done. The model has seen every training image once.

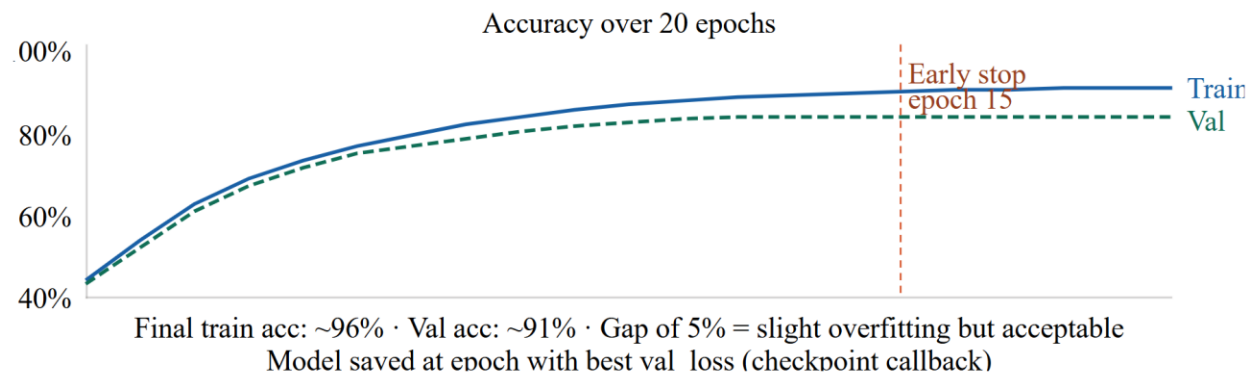
Epoch 1 progress



What you see in training logs: Each row is one epoch. You want val_loss to drop each epoch. If val_loss stops dropping but train_loss keeps dropping — that's overfitting. EarlyStopping watches val_loss and stops training before overfitting takes hold.

Step 7 — 20 epochs: the full training loop completes

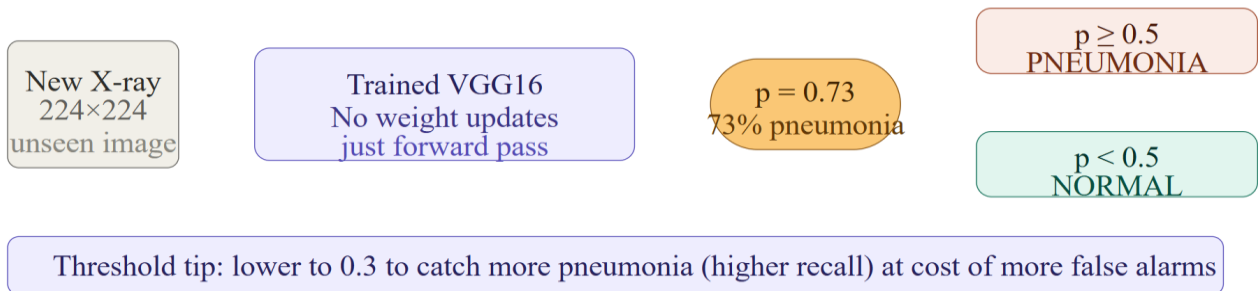
This entire cycle (all 163 batches) repeats 20 times. Each epoch, the model gets a little better. Training accuracy and validation accuracy both rise. EarlyStopping may end training early if the model stops improving.



How to read this graph in interviews: If train accuracy >> val accuracy (large gap) = overfitting. If both are low = underfitting. If both are high and close = good model. ModelCheckpoint saves the version with the best val_loss, not the last epoch.

Step 8 — Prediction on a new X-ray (inference)

Training is done. Now a brand new X-ray comes in — one the model has never seen. It goes through the same forward pass, produces a probability, and we apply a threshold (0.5) to give the final diagnosis.



Interview gold: The threshold 0.5 is not sacred. In a hospital setting you'd lower it to 0.3 so the model flags more cases for a radiologist to check. Missing pneumonia (FN) is far worse than a false alarm (FP). The threshold is a clinical decision, not just a technical one.